

FallGuard: Individual Contributions and Reflections

Yaorong Huang, Jiahao Zhao, Chengyi Xu, Zhonglin Chang, and Jiayun Song
Group 7, COMP6131 Fundamentals of the Internet of Things
Faculty of Applied Sciences, Macao Polytechnic University, Macao SAR, China

Abstract—This document consolidates the five members’ contribution statements and team task records for FallGuard, an edge-cloud fall-detection project using quality-aware bidirectional temporal modeling. It records individual work, collaboration, corresponding project deliverables, and learning outcomes. The contributions cover model development and deployment, data preparation, training, cloud delivery, interface development, testing, literature review, and project communication.

I. PROJECT AND TEAM ORGANIZATION

The final system extracts poses on a Raspberry Pi, classifies one-second skeleton windows with MaskedBiMamba, stores records locally, and synchronizes them to a persistent cloud archive. The descriptions below use the final system terminology; repository paths identify corresponding deliverables.

II. YAORONG HUANG

A. Role and overall responsibility

I served as group leader and principal implementation and integration contributor. I carried out most of the project’s code implementation and took primary responsibility for the technical manuscript and web presentation. My work connected the research model to a running IoT system: model design and preprocessing, experiment tooling, portable inference, Raspberry Pi deployment, local persistence, cloud synchronization, operator workspaces, and reproducibility materials. I also coordinated the interfaces between the team’s data, training, cloud, and reporting tasks and completed the final integration and consistency checks.

B. Model design, implementation, and experiment tooling

I implemented MaskedBiMamba, combining temporal attention, independent forward and backward Mamba processing, training-time frame masking, and quality-weighted pooling. I defined how normalized skeleton coordinates and joint confidence enter the temporal model and how the separate quality features determine frame weights. This required implementing the input representation, branch alignment, feature fusion, pooling, and classification interface, then connecting those components to the training and evaluation code. The resulting model provides a common input/output contract for research evaluation and deployment.

I implemented and organized the model-comparison and component-ablation tooling, including configuration files, random seeds, checkpoint selection, metric calculation, and result aggregation. The completed evaluation contains 60 primary runs across five models, four folds and three seeds; 120 ablation runs across ten variants; and nine runs in the

separate TCNTE reproduction track. My work covered the complete comparison pipeline, from recorded configurations and checkpoint selection to aggregated results and figures. I also organized primary robustness evaluation, external Le2i evaluation, and replay of the deployed confirmation rules.

I connected experiment outputs to the manuscript’s tables and plots, checking the distinction between window predictions, video predictions, merged events, and confirmed alerts. The evaluated model achieves the highest primary window precision at the common threshold (0.6048). Under 30% frame removal it retains window F1 of 0.6276, or 98.1% of its clean score, compared with 0.4361 for TCNTE. These results gave the model-development work a clear evaluation outcome and a concrete rationale for its deployment.

C. Model export and independent Raspberry Pi inference

I converted the selected temporal checkpoint into a portable two-input ONNX model and maintained its input shapes, normalization, class order, model card, and checksums. I implemented the portable inference path so that the Pi classifier does not depend on the CUDA training extensions. The deployment accepts a 30-step skeleton sequence and a separate three-feature quality sequence, matching the model used in the experiment pipeline.

I integrated USB-camera and video acquisition, YOLOv8s-pose extraction accelerated by Hailo, a CPU pose fallback, and local ONNX classification. I implemented timestamped one-second windows advanced every 0.2 seconds, interpolation, pose-quality checks, and handling of missing or non-monotonic observations. I also implemented the local alert transitions and connected them to stored records. This work turned the trained classifier into a service that can acquire input, classify windows, maintain state, and preserve results on the Raspberry Pi without an external network.

I supported the paired hardware evaluation and incorporated its measurements into the deployment and report. The complete local pipeline reached 20.56 frames/s with Hailo versus 1.73 frames/s with CPU pose extraction under the same YOLOv8s-pose comparison. Temporal classification remained on the Pi CPU. The deployed result therefore reflects acceleration of the complete implemented pipeline, together with a usable CPU fallback.

D. Persistent delivery and complete operator workspaces

I integrated SQLite record storage, an independent out-box, and cloud synchronization. The design preserves record identifiers and original capture times, commits cloud records

before acknowledgement, and safely retries pending records after reconnection. I connected these components to the cloud archive and the private device-status bridge. The recorded delivery tests matched all 85 evaluated records; a 30-second upload interruption retained 17 records, with the queue first observed empty 2.83 seconds after recovery.

I completed the final three-page local and cloud workspaces: detection overview, history, and device diagnostics. Beyond a basic prediction display, I implemented live preview, result freshness, historical probability charts, filters, pagination, CSV export, queue status, and resource diagnostics. I added explicit handling for incomplete poses, expired results, paused cameras, and lost device connections, so that the current display follows the operating state while saved history remains available.

I verified service startup and shutdown, archive queries, authentication, delivery, and recovery using the software suite and the actual page scripts. Deployment updates preserved existing SQLite data, checked service health and source hashes, and retained rollback copies. I also restored the browser-camera demonstration alongside the offline workspace, including its landmark-to-classifier adapter and current-model inference. This provided both the deployed Pi workflow and a computer-based demonstration entry.

E. Problems encountered and solutions implemented

One integration problem was maintaining the same model input in training and deployment. I made the pose normalization and separate quality input explicit, aligned the timestamped buffer with the training window definition, and checked preprocessing and PyTorch/ONNX numerical agreement. The two-input export comparison showed a maximum absolute logit difference of approximately 3.6×10^{-7} .

A second problem was keeping inference and recorded results available during network interruption. I separated camera inference, local storage, the outbox, and synchronization, then exercised restart, retry, and lost-acknowledgement behavior. UUID-based deduplication and acknowledgement after durable storage preserved the recorded history during recovery.

A third problem was that the earlier interface did not provide enough information for independent operation and troubleshooting. I replaced it with the full local/cloud workspaces and added freshness checks, history inspection, and diagnostics. I also corrected the preprocessing order in the missing-joint test path and updated the result selection, figures, and manuscript together. Resolving these issues required changes across model code, deployment services, interfaces, and documentation.

F. Manuscript, presentation, and reproducibility publication

I took primary responsibility for writing and revising the IEEE LaTeX report: method notation and equations, architecture diagrams, experiment protocols, results, system implementation, figures, captions, and references. I revised the argument around the model's precision, frame-loss tolerance, edge throughput, and recoverable cloud delivery, while keeping the

numerical statements consistent with the saved results. I also maintained the Chinese mathematical and system-workflow guides and the project brief, providing explanations for both technical review and presentation preparation.

I developed the twelve-slide GitHub Pages presentation, its interactive comparison charts, the searchable archive of 1,853 aggregate result rows, bilingual speaker notes, projection controls, presenter console, and rehearsal timer. I synchronized the data, explanations, source files, and published outputs across the report, guides, presentation, deployed software, and repository.

I organized the public reproducibility release: experiment and deployment source, dependency files, generic configuration and service templates, selected weights, ONNX model, asset hashes, dataset provenance, and run instructions. I used the 30-test software suite, numerical export checks, release manifests, and GitHub verification workflow to check the handover. I prepared the final report source, independent contribution document, presentation PDF, and fixed repository submission version as a coherent set of deliverables.

G. Deliverables and reflection

Main deliverables: `code/src/fallbench/`, `code/configs/`, `code/scripts/`, `deployment/`, `artifacts/models/`, `webui/`, `tests/`, `results/`, `manuscripts/final-report/`, `documentation/zh/`, `docs/`, and the reproducibility/submission guides.

My contribution extended from algorithm implementation to the final running system and its submitted evidence. The most demanding part was keeping these layers consistent: the model input used by training, the exported classifier, the device buffer, the cloud record, and the value shown to an operator. Working through that chain strengthened my ability to debug across components and to connect a research result to a maintainable IoT implementation. I also learned to treat experiment organization, recovery behavior, technical writing, and reproducibility as substantive engineering work, with concrete outputs that can be inspected and run.

III. JIAHAO ZHAO

A. Cloud interfaces and record delivery

My responsibilities covered the cloud interface, system testing, and project records. I worked on the interface through which classified edge records become a durable cloud history, including the record structure, storage behavior, and API access used by the detection interface. In the final system, temporal classification runs on the Pi and the cloud receives already classified records. The cloud-side workflow preserves the original observation time and acknowledges an identifier after the record has been stored, supporting safe retry after an interrupted transfer.

I contributed to the archive functions used for historical inspection: querying stored windows and events, filtering, pagination, and exporting records. I coordinated the device, timestamp, prediction, and alert fields with the edge implementation and the interface so that the recorded history and the displayed history describe the same data.

B. Testing and project coordination

I contributed system and regression tests for the record path, including malformed input handling, duplicate delivery, acknowledgement loss, restart persistence, and recovery after disconnection. The test evidence also covers archive access and the behavior of the actual service launchers. These checks supported the final demonstration by verifying the interaction between components, rather than assessing the classifier in isolation.

I maintained the consolidated meeting records, recording decisions, task owners, deadlines, and module handovers. Eight meetings between 9 September and 3 October documented the team’s progress. Meeting note-taking rotated among members, and I organized these records into the final set. I also collected service instructions, environment information, and test outputs as part of the reproducibility materials.

C. Problem and solution

A cloud write can succeed even when its acknowledgement is lost, leaving the edge to retry a record already stored. I worked with edge integration on commit-before-acknowledgement and UUID-based deduplication, and contributed retry and restart tests that checked for duplicate rows and preserved capture times.

D. Collaboration, deliverables, and reflection

I coordinated the record schema with Yaorong Huang and the displayed fields with Chengyi Xu, supplying system details for the report review.

Corresponding deliverables: `deployment/cloud/`, `webui/server.py`, `webui/cloud_workspace.py`, `tests/`, `results/deployment/`, and the group’s meeting records.

This work strengthened my understanding of durable delivery and service testing. Keeping storage and acknowledgement in the correct order, and exercising restart and retry behavior, turned recovery into a behavior we could demonstrate. Recording module handovers also helped the team resolve interface changes efficiently.

IV. CHENGYI XU

A. Training and experiment management

My responsibilities covered the training workflow and detection interface. I contributed the workflow that reads prepared skeleton sequences, forms training batches, addresses class imbalance, and manages checkpoints and metric logs. I worked with Yaorong Huang on the connection between the model definition and its training configuration, and with Jiayun Song on the input sequence format and grouped evaluation partitions.

I contributed training augmentation and random-seed management so that the comparison runs followed recorded settings. The final protocol trains for a fixed number of epochs, selects checkpoints by validation F1, and evaluates the same folds and seeds across models. The loss configuration, run commands, checkpoints, and metric outputs form the record of each experiment. I also supported the component comparisons

and the organization of their results into the report’s tables and figures.

B. Detection interface and demonstration

I contributed to the detection interface displaying device health, current predictions, and stored alerts, and to connecting those views to the cloud API. I worked on the initial interface panels and demonstration workflow. Yaorong Huang subsequently integrated and completed the final three-page local and cloud workspaces. This division connected my interface preparation with the final deployment and presentation.

I coordinated interface fields with Jiahao Zhao and checked that the displayed data matched stored records. I supported the demonstration material and the explanation of the detection workflow, including how an operator moves from current status to historical inspection. These activities made the project accessible to viewers who were not involved in model development.

C. Problem and solution

Fall windows were much rarer than daily-activity windows, creating a risk that training would favor the majority class. I used class-aware weighted focal-loss settings and recorded masking augmentation and random seeds in the shared workflow, so the comparison runs followed the same class-balancing and run protocol.

D. Collaboration, deliverables, and reflection

I coordinated data inputs with Jiayun Song, model settings with Yaorong Huang, and interface fields with Jiahao Zhao.

Corresponding deliverables: `code/src/fallbench/engine.py`, `code/src/fallbench/losses.py`, `code/src/fallbench/reproducibility.py`, `code/scripts/train.py`, `code/configs/`, and the detection-interface and demonstration materials.

I learned to treat configuration and logging as part of the experiment, rather than as supporting paperwork. Shared inputs, consistent run settings, and saved outputs made model comparisons easier to interpret. Interface work also showed me the value of displaying status, time, and history together when explaining a running system.

V. ZHONGLIN CHANG

A. Problem definition and literature review

I took responsibility for the problem definition, algorithm survey, and references. I framed the task around distinguishing a fall transition from daily activities with similar individual poses, such as sitting, bending, or kneeling. This motivated the use of temporal skeleton sequences and the explicit consideration of incomplete pose observations. I reviewed wearable, environmental, and camera-based approaches before focusing the comparison on pose-based temporal modeling and edge deployment.

I organized the candidate methods and their uses in the project. The review included TCNTE as a temporal baseline, masked temporal modeling as a motivation for frame masking, and confidence-aware skeleton approaches as context for the

quality input. I helped articulate MaskedBiMamba’s contribution as the combination of temporal context and pose quality, together with matched evaluation and practical edge–cloud integration.

B. References and report review

I maintained the shared reference list and checked the sources used in the report. The literature records connect the cited methods and datasets to their source documents and distinguish available implementation resources from papers used as methodological background. I contributed the problem-definition and related-work material and reviewed the alignment between the report’s claims and the team’s recorded results.

I coordinated the method, dataset, and system descriptions with the team to keep the introduction, method, and results consistent.

C. Problem and solution

Posture or motion thresholds alone can confuse rapid sitting with a fall, so selecting a model required a clear account of the temporal information it uses. I compared pose-based temporal candidates under the project’s camera and compute constraints, checked source papers and implementation resources, and wrote the selection rationale used in the problem definition and related work.

D. Deliverables and reflection

Corresponding deliverables: `manuscripts/final-report/texfiles/2.Introduction.tex`, `manuscripts/final-report/references.bib`, `manuscripts/proposal/REFERENCE_VERIFICATION.md`, and the candidate-method and reference notes.

I learned that a useful literature review must connect a research idea to an implementable comparison. Checking datasets, code availability, and the precise purpose of each citation helped the team choose practical baselines and explain its design clearly. Reviewing the report against the result records also improved the consistency of the final argument.

VI. JIAYUN SONG

A. Data preparation and evaluation inputs

My responsibilities covered dataset preparation, edge testing, and presentation materials. I organized dataset access information, labels, usage terms, and the prepared skeleton sequences used by the training workflow. The final primary evaluation uses CAUCAFall and GMDCSA-24, while Le2i provides a separate baseline comparison and an external test. I coordinated the input sequence format and the evaluation partitions with Chengyi Xu and Yaorong Huang.

I contributed to label mapping, subject-grouped partition preparation, and the conversion of videos into reusable pose sequences. Keeping a subject’s recordings within one partition supports a fair cross-subject comparison, and preparing the sequence once lets training reuse the same representation. The final materials publish the recovered test assignment for the 260 primary videos together with its provenance information.

B. Edge testing and presentation preparation

I contributed edge measurements and the organization of robustness and cross-dataset results. The paired hardware measurements compare Hailo and CPU pose processing at the same input resolution while retaining the same local classifier. I supported the collection and presentation of throughput, processing latency, memory, and temperature evidence, as well as the preparation of the synthetic edge-evaluation sample.

I prepared presentation content, figures, and speaker notes, helped maintain a consistent visual style, and supported rehearsal. Yaorong Huang implemented the final interactive web presentation; my work focused on communicating the datasets, system workflow, and evaluation results within that shared presentation. I also supported the preparation of demonstration and non-fall example material described in my individual work statement.

C. Problem and solution

The CPU pose path provided limited throughput for the intended edge workflow. I helped measure Hailo and CPU processing at the same input resolution using a paired replay; the archived full-pipeline results were 20.56 versus 1.73 frames/s, providing evidence for the accelerated deployment configuration.

D. Collaboration, deliverables, and reflection

I coordinated training inputs with Chengyi Xu and edge measurements with Yaorong Huang, and supplied timestamps and citations for the report.

Corresponding deliverables: `datasets/DATASET_PROVENANCE_AND_USAGE.md`, `datasets/annotation_sources.json`, `datasets/primary_test_folds.csv`, `code/scripts/make_splits.py`, `code/scripts/build_windows.py`, `results/edge/`, `results/robustness/`, and the presentation content and notes.

I learned to inspect data preparation and partitions before interpreting model scores. Clear dataset roles, recorded transformations, and consistent measurements made the comparisons easier to explain. Preparing the presentation also helped me turn technical results into a coherent account of how the data, model, and deployed system work together.